

Définir Agile

Les études de Waterfall, XP, Scrum nous aide à définir Agile

01 - Rappels

Rappels des cours Naissance Agile, XP et Scrum

Naissance Agile : L'échec de « Waterfall »

- La mauvaise interprétation de Waterfall a conduit à vouloir analyser complètement en amont. => Echec de Waterfall :
 - Il n'existe pas un ensemble fini de besoins
 - Les changements seront petits
 - On arrive à délivrer dans les temps
 - On définit plusieurs fois le logiciel

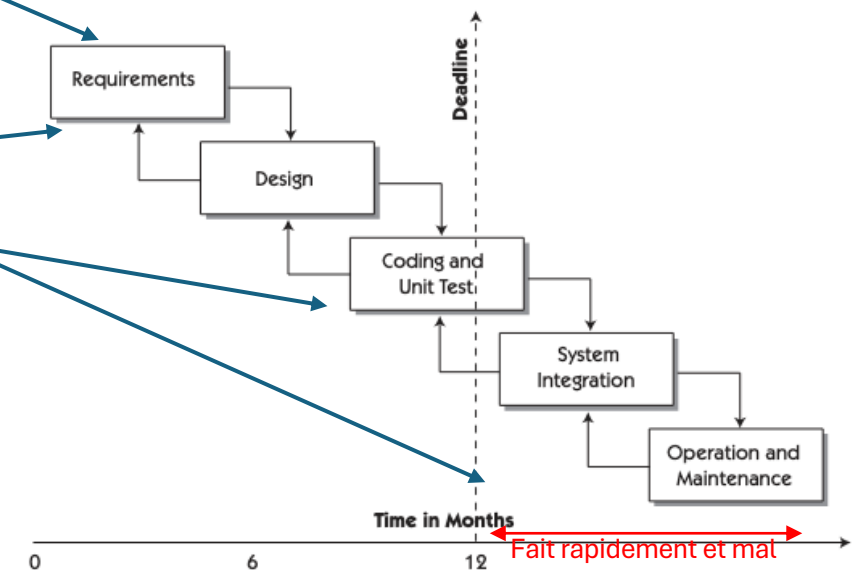


Figure 2-3 What happens when everything except the deadline moves to the right

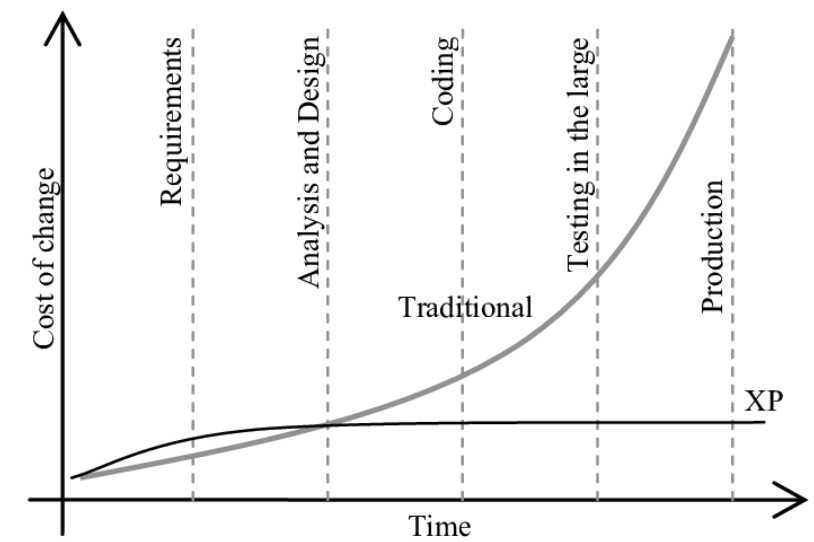
Framework XP

Concept clé : assurer un coût de changement $\log(x)$

- Des valeurs, des principes et des pratiques pour avoir un **feedback régulier** et assurer une **excellence technique**

- Small Release
- On-site customer
- Whole Team
- TDD
- Continuous Integration

...



« At its root, Scrum is based on a simple idea: whenever you start a project, why not regularly check in, see if what you're doing is heading in the right direction, and if it's actually what people want? »

Framework Scrum

Concept clé : Inspect & Adapt

- Créer une équipe Scrum (The new new Product Dev. Game)
 - Autonome
 - Pluridisciplinaire
 - Une identité (Vision transcendante)
 - Est stable
- => Les artefacts Scrum servent à rendre le travail **visible et transparent**
 - Les artefacts engagement donne un objectif/repère
- => Les cérémonies assurent une régularité de **feedback réguliers (I&A)**

Points communs des frameworks

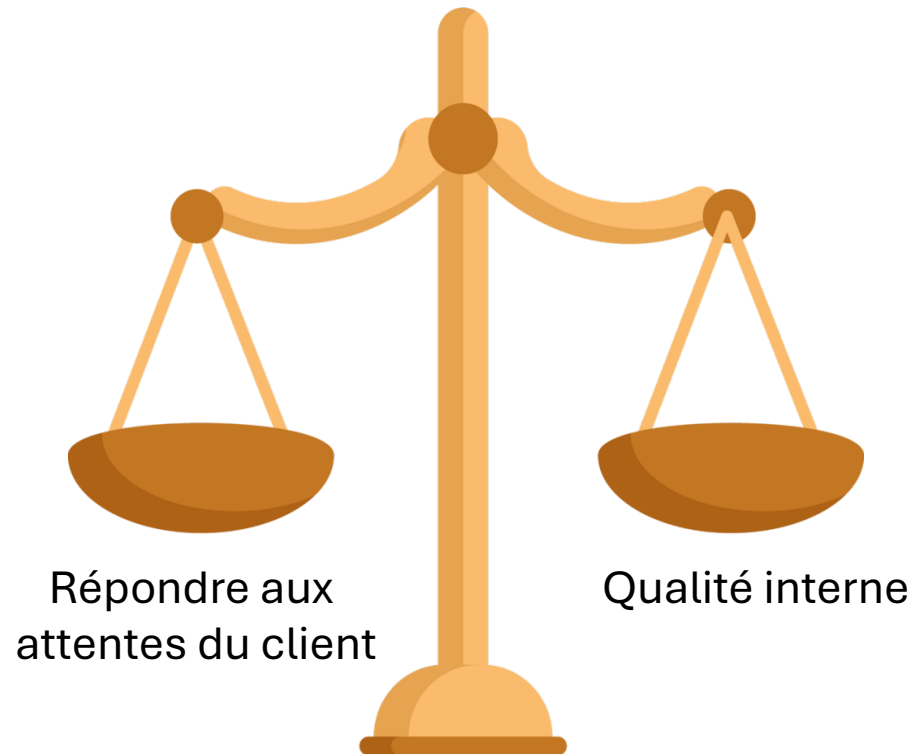
- Avoir des boucles de feedback rapide
 - COMMENT? : en inspectant et en s'adaptant
 - POURQUOI? : car le développement logiciel est incertain

02 – Pourquoi devenir Agile ?

Abstraction des concepts de XP et Scrum

1. Livrer un produit de haute qualité

Qu'est-ce un produit de haute qualité ?



Affirmation : Viser l'excellence

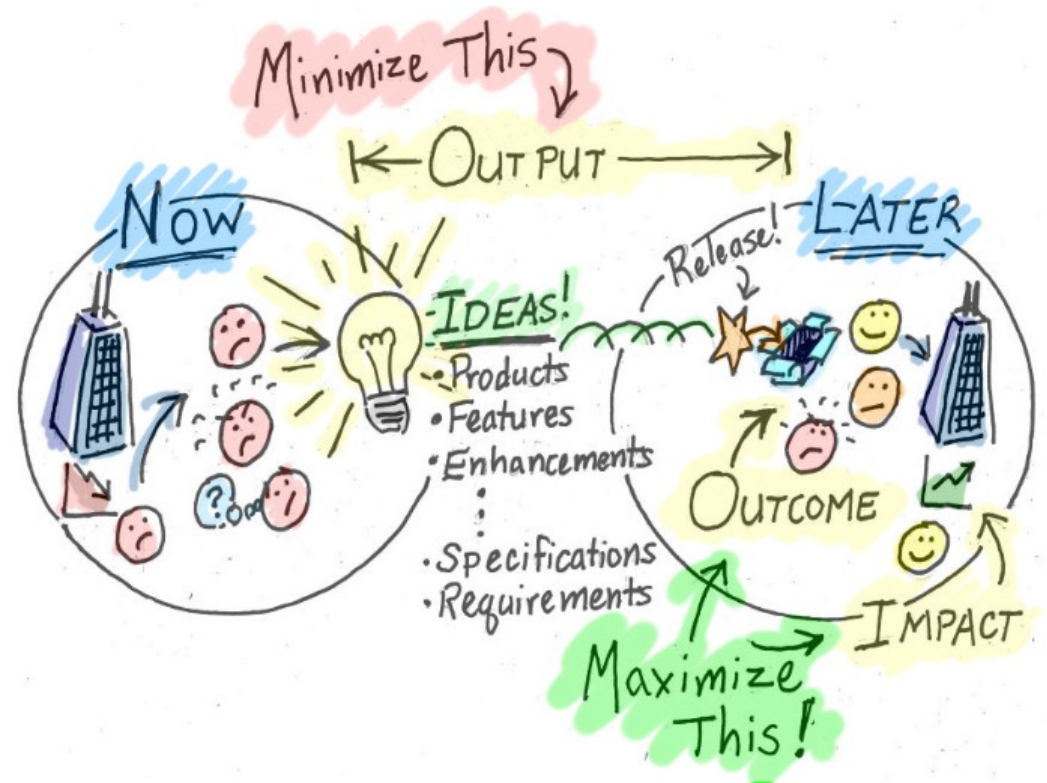
Agile est un engagement à tendre vers l'excellence, à être toujours plus professionnel et à favoriser les comportements professionnels dans l'industrie logiciel

1.1 Livrer un produit de haute qualité : d'un pdv business

Affirmation

Notre objectif est de créer de la valeur pour le client

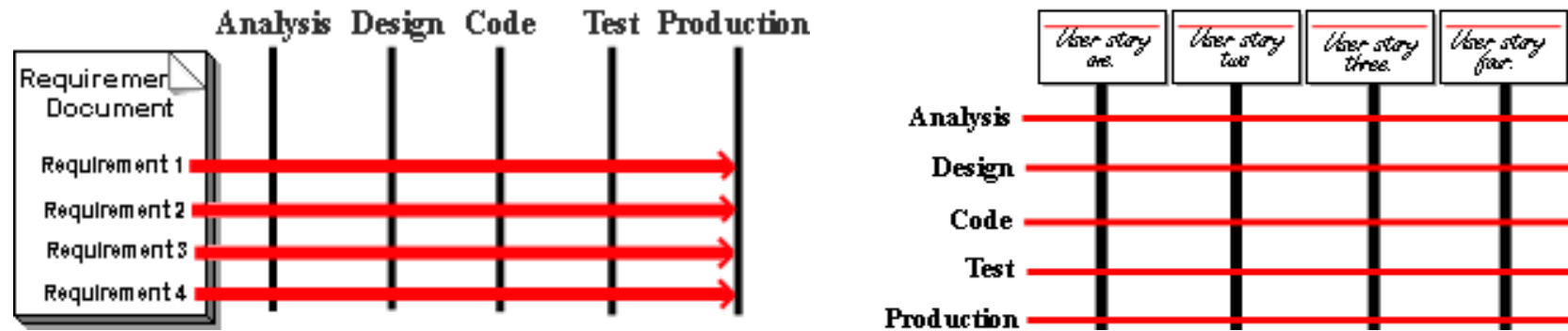
- Output = fonctionnalités livrées
Mais on ignore si utilisée ...
- Outcome = l'impact d'une fonctionnalité



1.1 Livrer un produit de haute qualité : d'un pdv business

Comment créer de la valeur métier : Feature > Activité

1. Feature > Activité



- Car la valeur existe uniquement lorsque la feature est en production

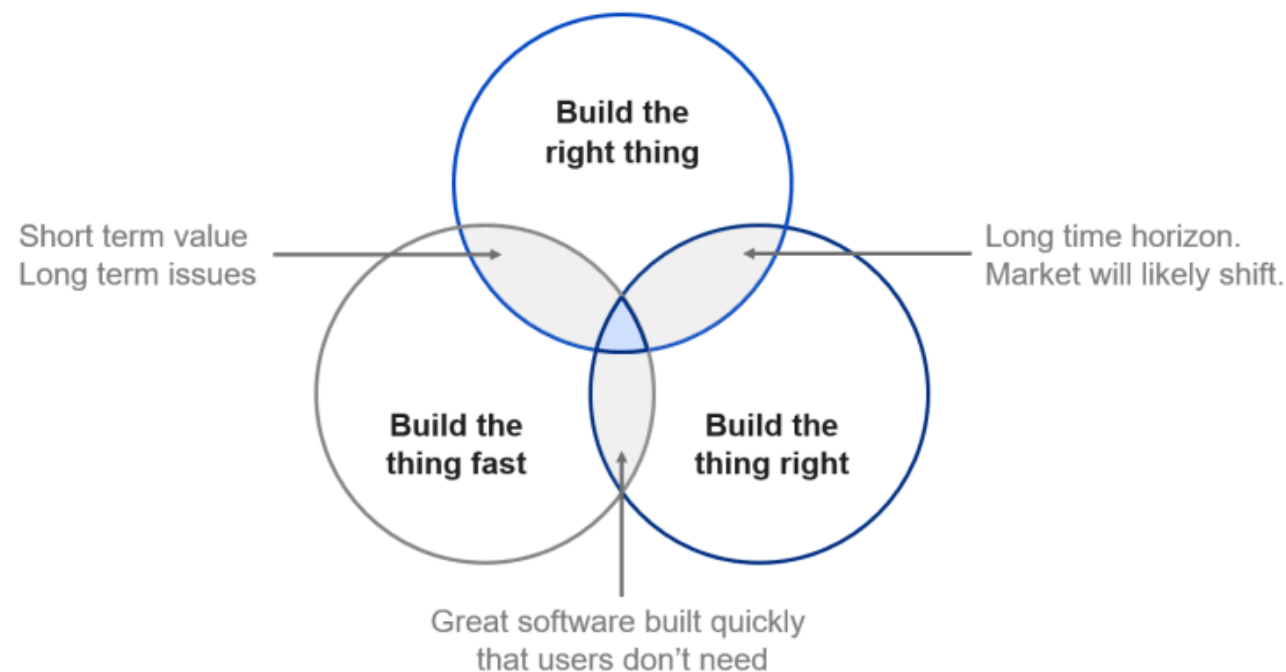
2. Prioriser

- Nous ne pourrions pas implémenter toutes les features, il faut donc les prioriser (cf Chapitre Estimation et Planification)

3. Implication du client

1.1 Livrer un produit de haute qualité : d'un pdv business

Comment créer de la valeur métier : implication du client

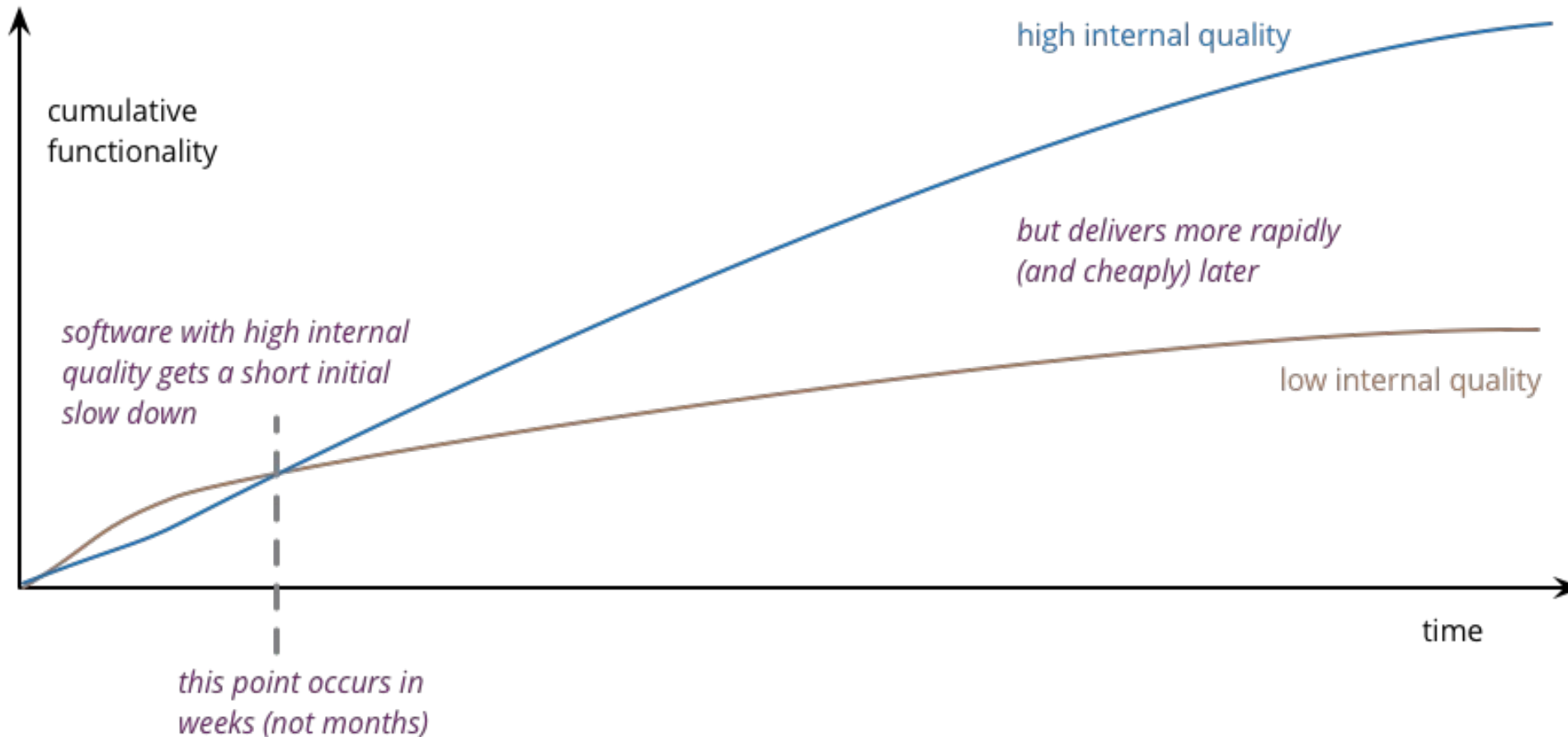


	Question	Remède
Build the thing right	Le produit est-il bien construit ?	La qualité, les tests, la Definition of Done
Build the right thing	Est-ce le bon produit ?	L'engagement du client, le feedback, la mesure d'usage

Les personnes en charge du métier ou des affaires et les personnes en charge de la réalisation doivent travailler ensemble chaque jour, tout au long du projet.
(principe 4 du manifeste agile)

1.2. Livrer un produit de haute qualité : d'un pdv technique

Pourquoi la qualité interne d'un logiciel est importante ?



The fundamental role of internal quality is that it lowers the cost of future change.

Is High Quality Software Worth the Cost?

Martin Fowler (<https://martinfowler.com/articles/is-quality-worth-cost.html>)

1.2. Livrer un produit de haute qualité : d'un pdv technique

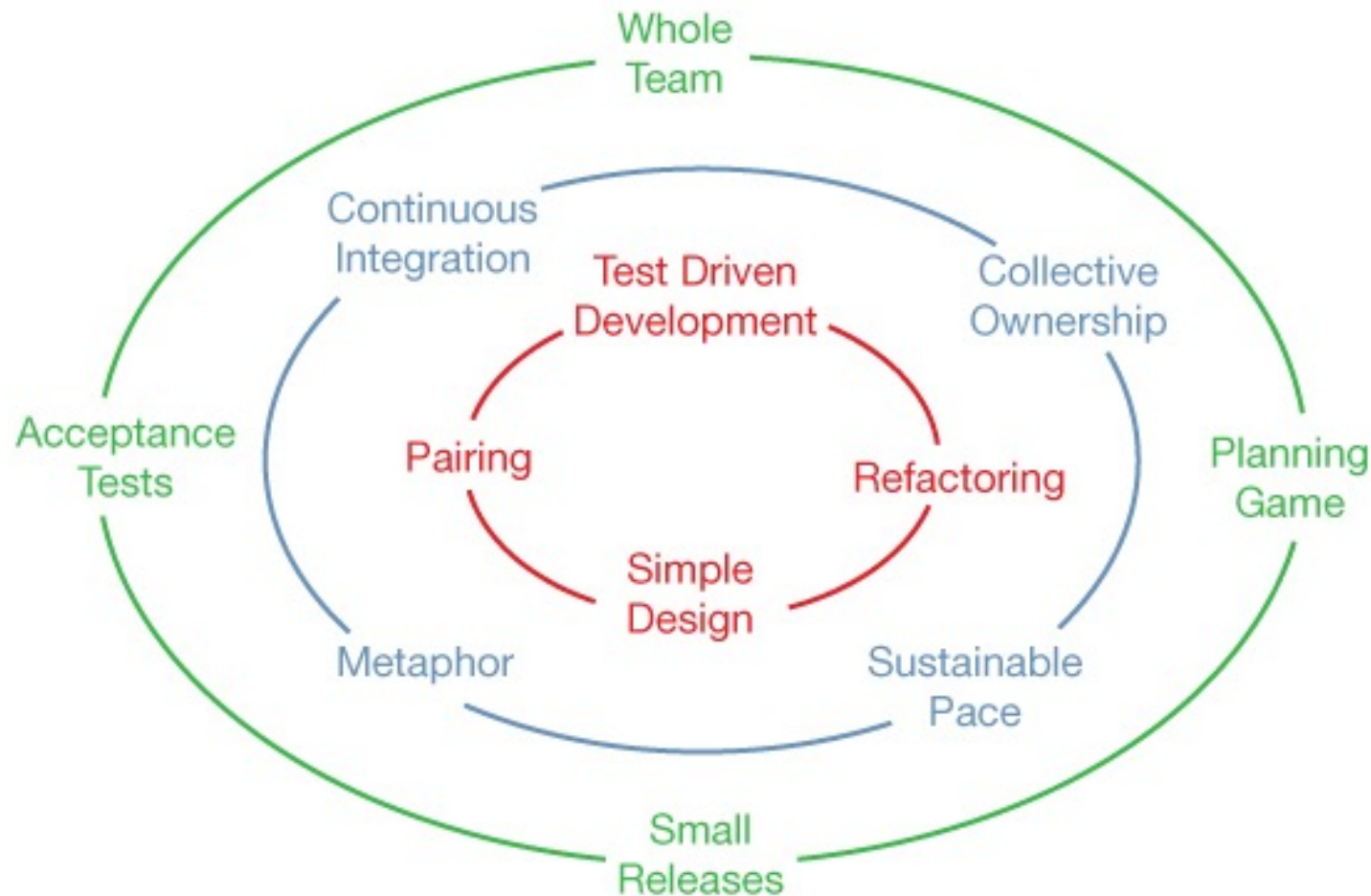
Pourquoi la dette technique augmente le coût logiciel ?

- Augmentation du *Lead Time*
- Augmentation du coût de développement et de support
 - une fonctionnalité qui pourrait être finalisée avec un petit budget aujourd'hui coûte cher et son implémentation peut se voir annulée.
- Non prédictible
 - Lorsque la dette est trop élevée les estimations (coût et délais) deviennent compliquées à chiffrer ou sont complètement fausses
- Frustration
 - l'accumulation de ces petits problèmes rend le travail sur le produit pénible (dette de motivation)

1.2. Livrer un produit de haute qualité : d'un pdv technique

Comment assurer une qualité interne : l'excellence technique

Agile = Gdp + Technique



1.2. Livrer un produit de haute qualité : d'un pdv technique

Comment assurer une qualité interne : l'excellence technique

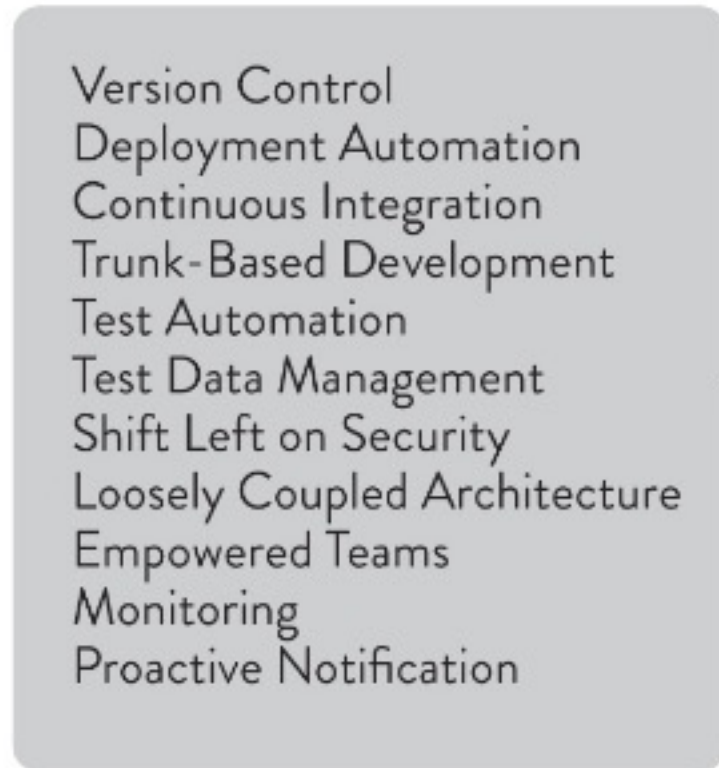


Figure 4.1: Drivers of Continuous Delivery

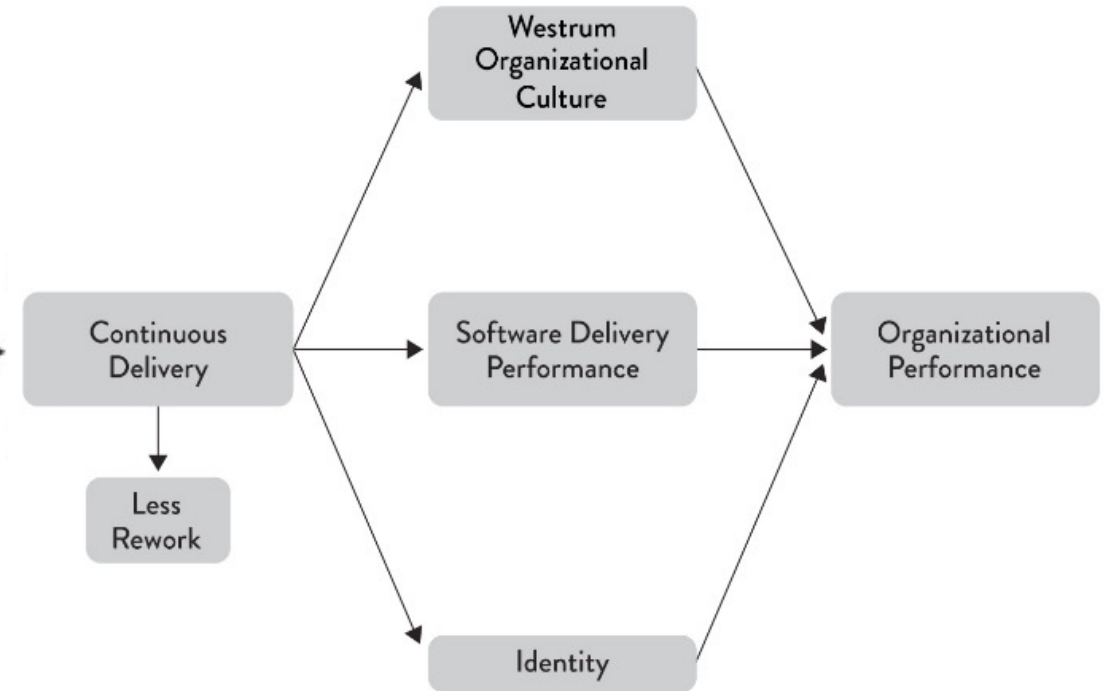


Figure 4.2: Impacts of Continuous Delivery

L'excellence technique impacte les performances de livraison

2. Accepter une part d'incertitude

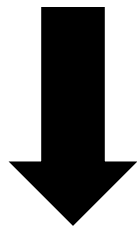
D'où vient l'incertitude ?

- Le client ne connaît pas ses besoins
- Le développement logiciel est un process créatif se qui le rend compliqué à planifier et donc prédictible

2. Accepter une part d'incertitude

Controlling an Unpredictable Process – Iterations <https://www.martinfowler.com/articles/newMethodology>

- Agile cherche à savoir le plus tôt possible à quel point on part dans la mauvaise direction, afin de garder le contrôle de la situation.



Via des boucles de feedback + itération

Affirmation

The loss of hope is a major goal of Agile.

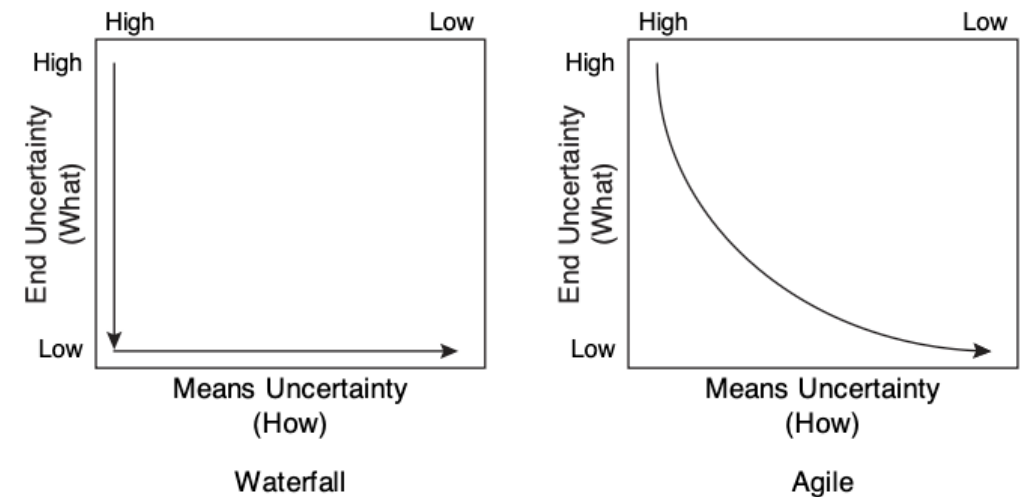


Figure 9.1 Traditional and agile views of reducing uncertainty. Adapted from Laufer (1996).

Waterfall : un produit entièrement spécifié dès le début

Agile : on sait qu'il est impossible en début de projet d'éliminer TOUTE l'incertitude => Pour la réduire on utilise boucle de feedback

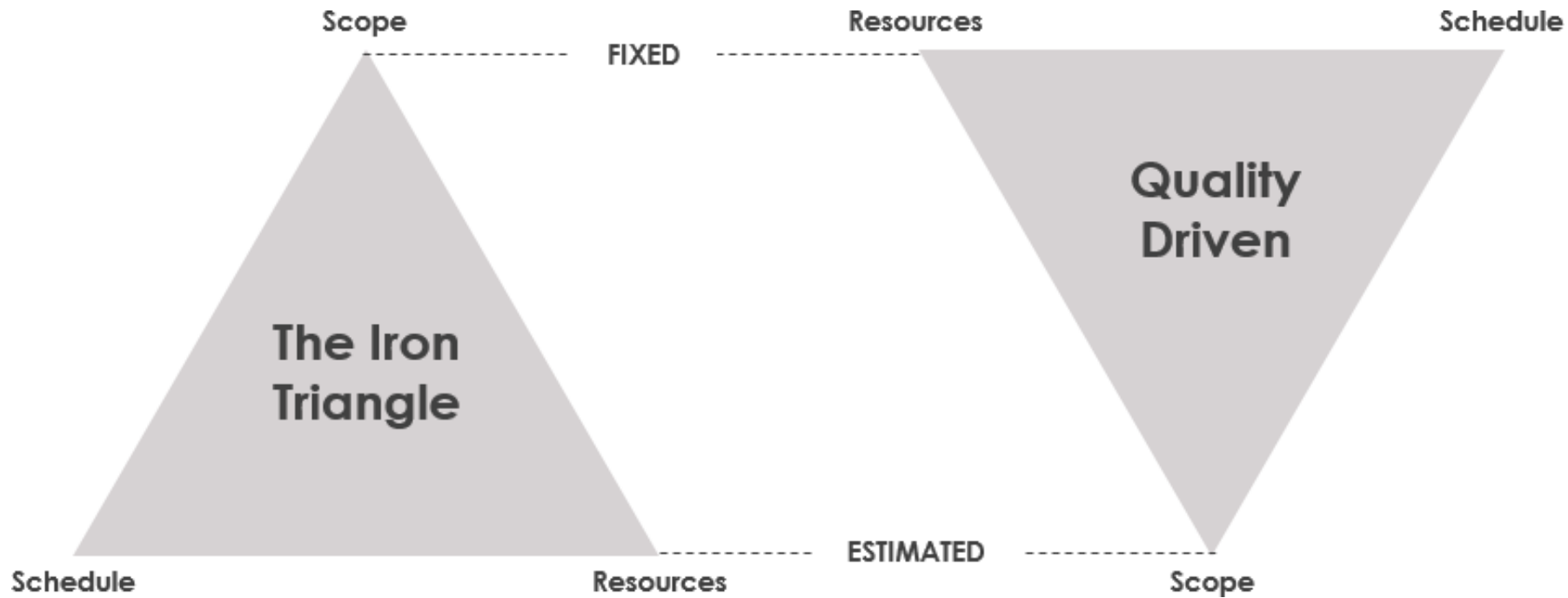
2. Accepter une part d'incertitude

Des itérations courtes

- Permettent d'apporter du contrôle dans une situation peu prédictible
- MAIS nécessite de changer la relation contractuel DEV/Client par une relation de **collaboration**
- => DONC à changer notre relation aux variables dynamiques et fixes

2. Accepter une part d'incertitude

Pour assurer des itérations courtes notre seule variable ajustable est le périmètre



Agile

- La deadline + les ressources sont fixes
- Le **périmètre est variable**

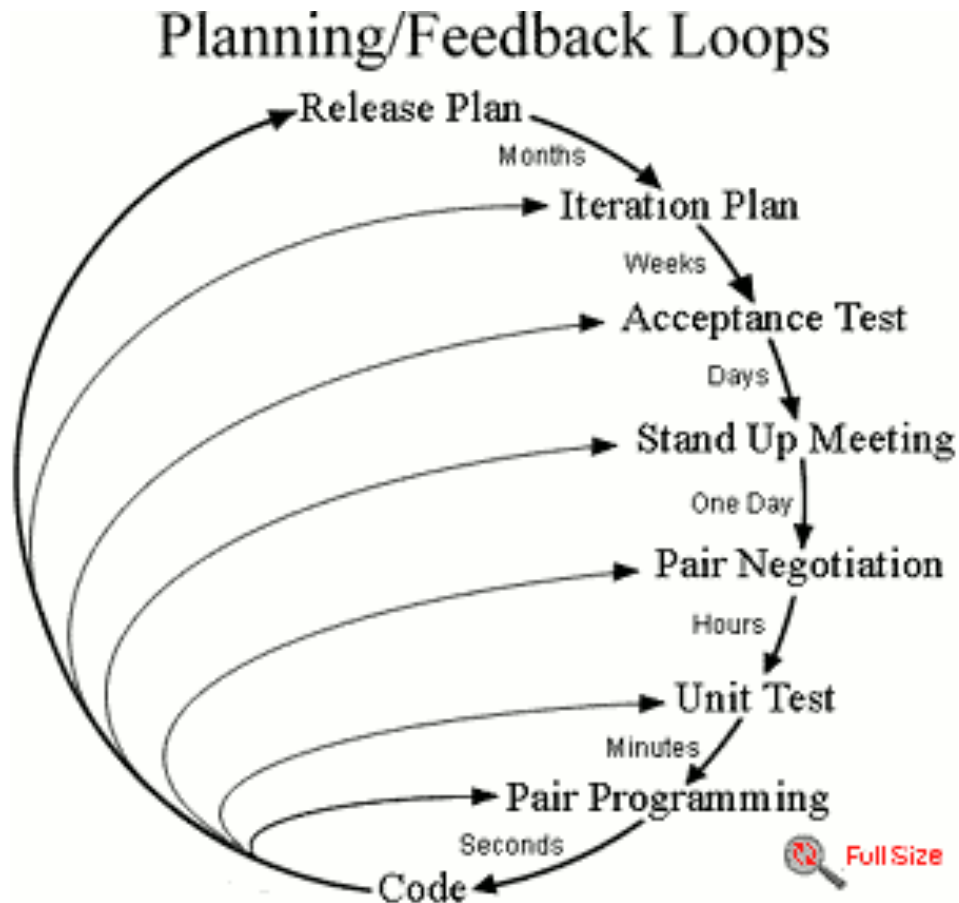
2. Accepter une part d'incertitude

Pour assurer des itérations courtes notre seule variable ajustable est le périmètre

- **Changer calendrier** : impossible
- **Ajouter des ressources** : Loi de Brooks
- **Réduire la qualité** : *Le seul moyen d'aller vite, c'est d'avancer proprement*
- **Changer le périmètre** : dans l'ordre de la valeur à produire -> implique prioriser

2. Accepter une part d'incertitude

Les boucles de feedback à plusieurs niveaux



Some folks think that Agile is about going fast. It's not. It's never been about going fast. **Agile is about knowing, as early as possible, just how screwed we are (à quel point nous sommes dans la merde).** – Robert C Martin

Feedbacks

Car couplés aux itérations ils permettent de **savoir au plus tôt** et la connaissance permet de nous adapter (Inspect & Adapt)

Niveau équipe : qualité logicielle, ambiance, etc ...

Niveau produit : **construit-on la bonne chose**

Remettre l'humain au centre du processus

Agile Development

is **adaptive** rather than predictive
is **people-oriented** rather than
process-oriented

<https://martinfowler.com/agile.html>



Les parties prenantes (stakeholders)



*I would like the world to be safe for
developers*

- Kent Beck

03 – La philosophie Agile

Définition de Agile



Le Manifeste Agile

2001

On February 11-13, 2001, at The Lodge at Snowbird ski resort in the Wasatch mountains of Utah, seventeen people met to talk, ski, relax, and try to find common ground—and of course, to eat. What emerged was the Agile 'Software Development' Manifesto. Representatives from Extreme Programming, SCRUM, DSDM, Adaptive Software Development, Crystal, Feature-Driven Development, Pragmatic Programming, and others sympathetic to the need for an alternative to documentation driven, heavyweight software development processes convened.

Now, a bigger gathering of organizational anarchists would be hard to find, so what emerged from this meeting was symbolic—a *Manifesto for Agile Software Development*—signed by all participants. The only concern with the term *agile* came from Martin Fowler (a Brit for those who don't know him) who allowed that most Americans didn't know how to pronounce the word 'agile'.

Alistair Cockburn's initial concerns reflected the early thoughts of many participants. "I personally didn't expect that this particular group of agilites to ever agree on anything substantive." But his post-meeting feelings were also shared, "Speaking for myself, I am delighted by the final phrasing [of the Manifesto]. I was surprised that the others appeared equally delighted by the final phrasing. So we did agree on something substantive."

Naming ourselves "The Agile Alliance," this group of independent thinkers about software development, and sometimes competitors to each other, agreed on the *Manifesto for Agile Software Development* displayed on the title page of this web site.

But while the Manifesto provides some specific ideas, there is a deeper theme that drives many, but not all, to be sure, members of the alliance. At the close of the two-day meeting, Bob Martin joked that he was about to make a "mushy" statement. But while tinged with humor, few disagreed with Bob's sentiments—that we all felt privileged to work with a group of people who held a set of compatible values, a set of values based on trust and respect for each other and promoting organizational models based on people, collaboration, and building the types of organizational communities in which we would want to work. At the core, I believe Agile Methodologists are really about "mushy" stuff—about delivering good products to customers by operating in an environment that does more than talk about "people as our most important asset" but actually "acts" as if people were the most important, and lose the word "asset". So in the final analysis, the meteoric rise of interest in—and sometimes tremendous criticism of—Agile Methodologies is about the mushy stuff of values and culture.

For example, I think that ultimately, Extreme Programming has mushroomed in use and interest, not because of pair-programming or refactoring, but because, taken as a whole, the practices define a developer community freed from the baggage of Dilbertesque corporations. Kent Beck tells the story of an early job in which he estimated a programming effort of six weeks for two people. After his manager reassigned the other programmer at the beginning of the project, he completed the project in twelve weeks—and felt terrible about himself! The boss—of course—harangued Kent about how slow he was throughout the second six weeks. Kent, somewhat despondent because he was such a "failure" as a programmer, finally realized that his original estimate of 6 weeks was extremely accurate—for 2 people—and that his "failure" was really the manager's failure, indeed, the failure of the standard "fixed" process mindset that so frequently plagues our industry.

This type of situation goes on every day—marketing, or management, or external customers, internal customers, and, yes, even developers—don't want to make hard trade-off decisions, so they impose irrational demands through the imposition of corporate power structures. This isn't merely a software development problem, it runs throughout Dilbertesque organizations.

In order to succeed in the new economy, to move aggressively into the era of e-business, e-commerce, and the web, companies have to rid themselves of their Dilbert manifestations of make-work and arcane policies. This freedom from the inanities of corporate life attracts proponents of Agile Methodologies, and scares the beegeebers (you can't use the word 'shit' in a professional paper) out of traditionalists. Quite frankly, the Agile approaches scare corporate bureaucrats—at least those that are happy pushing process for process' sake versus trying to do the best for the "customer" and deliver something timely and tangible and "as promised"—because they run out of places to hide.

The Agile movement is not anti-methodology, in fact, many of us want to restore credibility to the word methodology. We want to restore a balance. We embrace modeling, but not in order to file some diagram in a dusty corporate repository. We embrace documentation, but not hundreds of pages of never-maintained and rarely-used tomes. We plan, but recognize the limits of planning in a turbulent environment. Those who would brand proponents of XP or SCRUM or any of the other Agile Methodologies as "hackers" are ignorant of both the methodologies and the original definition of the term hacker.

The meeting at Snowbird was incubated at an earlier get together of Extreme Programming proponents, and a few "outsiders," organized by Kent Beck at the Rogue River Lodge in Oregon in the spring of 2000. At the Rogue River meeting attendees voiced support for a variety of "Light" methodologies, but nothing formal occurred. During 2000 a number of articles were written that referenced the category of "Light" or "Lightweight" processes. A number these articles referred to "Light methodologies, such as Extreme Programming, Adaptive Software Development, Crystal, and SCRUM". In conversations, no one really liked the moniker "Light", but it seemed to stick for the time being.

In September 2000, Bob Martin from Object Mentor in Chicago, started the next meeting ball rolling with an email; "I'd like to convene a small (two day) conference in the January to February 2001 timeframe here in Chicago. The purpose of this conference is to get all the lightweight method leaders in one room. All of you are invited; and I'd be interested to know who else I should approach." Bob set up a Wiki site and the discussions raged.

Early on, Alistair Cockburn weighed in with an epistle that identified the general disgruntlement with the word 'Light': "I don't mind the methodology being called light in weight, but I'm not sure I want to be referred to as a lightweight attending a lightweight methodologists meeting. It somehow sounds like a bunch of skinny, feeble-minded lightweight people trying to remember what day it is."

The fiercest debate was over location! There was serious concern about Chicago in wintertime—cold and nothing fun to do; Snowbird, Utah—cold, but fun things to do, at least for those who ski on their heads like Martin Fowler tried on day one; and Anguilla in the Caribbean—warm and fun, but time consuming to get to. In the end, Snowbird and skiing won out; however, a few people—like Ron Jeffries—want a warmer place next time.

We hope that our work together as the Agile Alliance helps others in our profession to think about software development, methodologies, and organizations, in new- more agile - ways. If so, we've accomplished our goals.

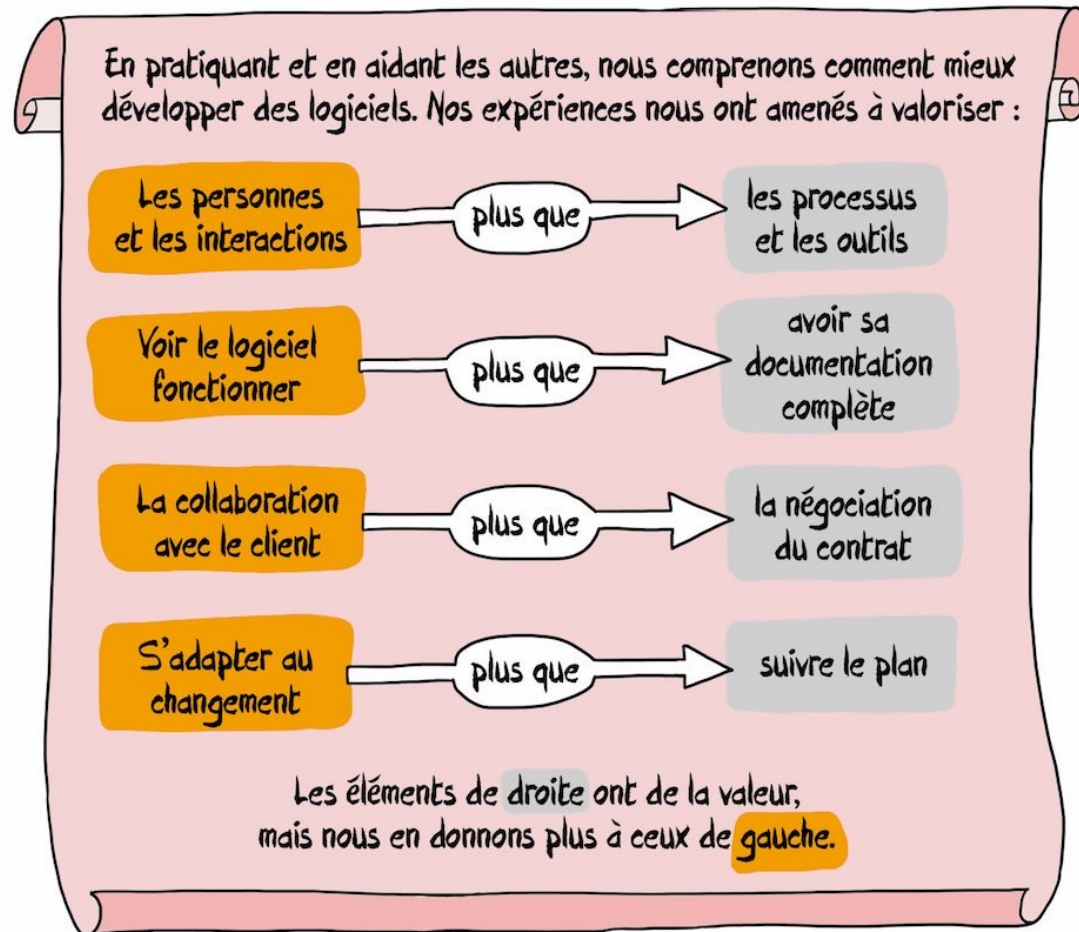
Jim Highsmith, for the Agile Alliance

©2001 Jim Highsmith

- De cette rencontre naît le **Manifeste Agile**, fondé sur des valeurs comme la **collaboration**, la **confiance**, l'**adaptation au changement et la priorité donnée aux personnes et aux clients**.
- L'Agile ne rejette pas totalement les méthodes, la documentation ou la planification, mais cherche à les utiliser de manière **simple et utile**, plutôt que de suivre des processus rigides.
- L'idée centrale est donc de créer des équipes capables de **s'adapter rapidement, collaborer efficacement et livrer régulièrement un produit utile au client**.

Les 4 valeurs du Manifeste Agile

Agile est une déclaration de quatre valeurs et de 12 principes.



Les 12 principes

- Notre principale priorité est de satisfaire le client en livrant rapidement et régulièrement des solutions qui apportent de la valeur.
- Accueillez chaleureusement les changements de besoins, même tardifs dans le développement. Les processus agiles tirent parti du changement pour renforcer l'avantage concurrentiel du client.
- Livrez souvent des solutions opérationnelles, à une fréquence allant de quelques semaines à quelques mois, avec une préférence pour les échelles de temps les plus courtes.
- Les personnes en charge du métier ou des affaires et les personnes en charge de la réalisation doivent travailler ensemble chaque jour, tout au long du projet.
- Construisez les projets à partir de personnes motivées. Donnez-leur l'environnement et le soutien dont elles ont besoin et faites-leur confiance pour mener à bien le travail.
- La conversation en face à face est la méthode la plus efficace et la plus économique pour donner des informations à une équipe de réalisation, et pour échanger des informations à l'intérieur de l'équipe.
- La disponibilité de solutions opérationnelles est la principale mesure d'avancement.
- Les processus agiles encouragent à respecter un rythme soutenable lors de la réalisation. Les commanditaires, les réalisateurs et les utilisateurs devraient pouvoir maintenir indéfiniment un rythme constant.
- Porter continuellement attention à l'excellence technique et à la qualité de la conception renforce l'agilité.
- La simplicité – l'art de maximiser la quantité de travail qu'on ne fait pas – est essentielle.
- Les meilleures architectures, les meilleures spécifications de besoins, et les meilleures conceptions émergent d'équipes auto-organisées.
- À intervalles réguliers, l'équipe réfléchit aux façons de devenir plus efficace, puis modifie son comportement et l'ajuste en conséquence.

« Mes » définitions

Agile

- C'est une philosophie
- Qui définit un ensemble de valeurs et de principes
- Pour réaliser un produit de haute qualité
- Dans un monde à forte incertitude

Agile

- adopter une approche itérative et incrémentale
- menée dans un esprit collaboratif
- pour générer un produit de haute qualité
- en tenant compte de l'évolution client
- **flexibilité, répétition, ouverture aux changements, feedback**

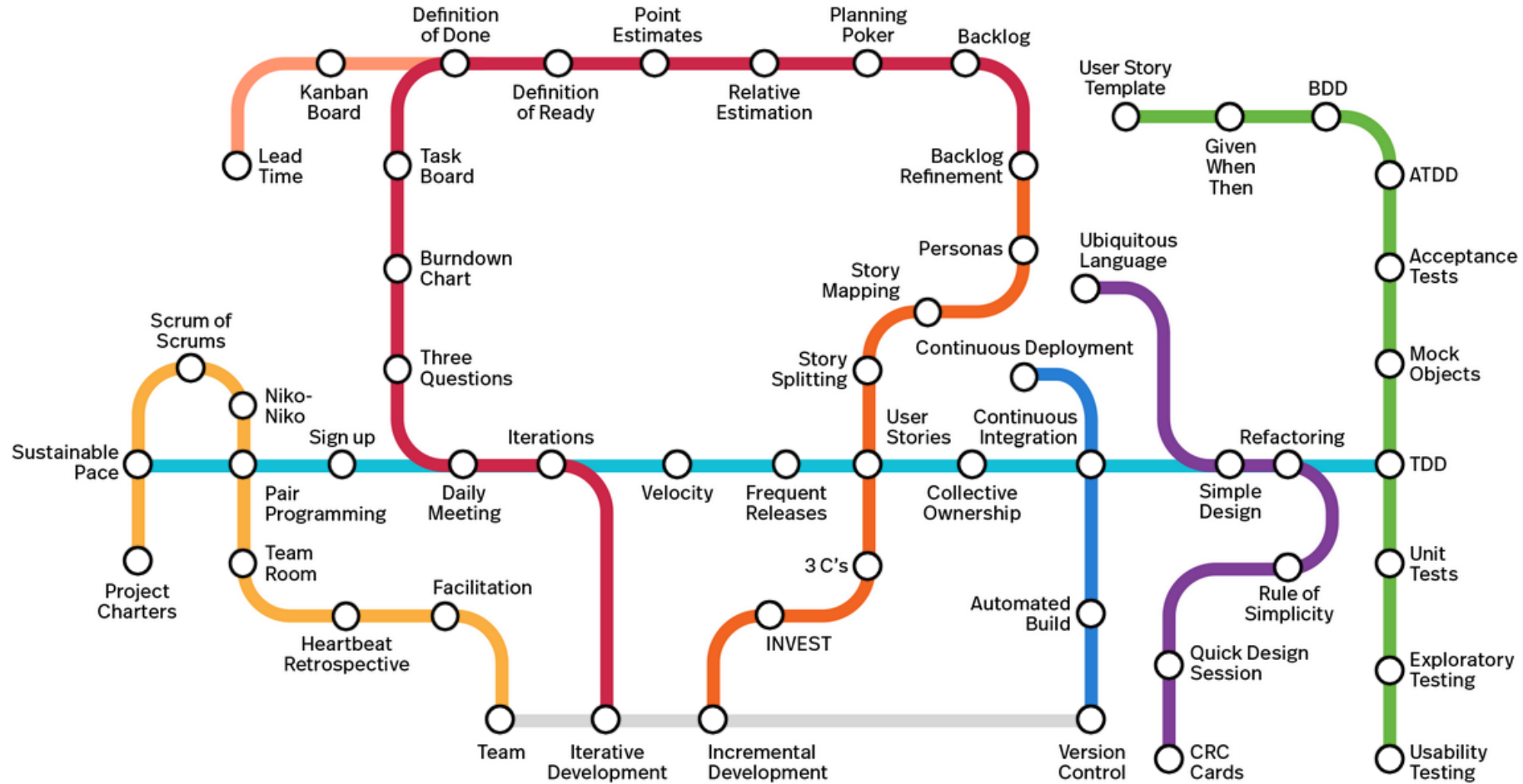
« Mes » Définitions complètes

- C'est une philosophie qui définit un ensemble de valeurs et de principes *QUOI*
- Pour réaliser un produit de haute qualité *POURQUOI*
- En adoptant une approche itérative et incrémentale menée dans un esprit collaboratif *COMMENT*
- Et en tenant compte de l'évolution client (monde à forte incertitude)

Des pratiques Agile

XP, SCRUM ont fait naître le Manifeste et les pratiques

The Agile Subway Map



The colored “subway” lines represent practices from the various Agile approaches or areas of concern.

Click on the “station” circles to read the full definition for each term.

— Extreme Programming	— Scrum	— Design
— Teams	— Product Management	— Testing
— Lean	— DevOps	— Fundamentals